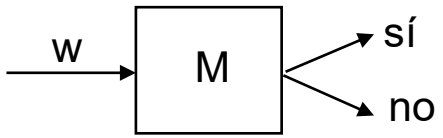
Clase teórica 7

La jerarquía espacio-temporal
y
los problemas de búsqueda revisitados

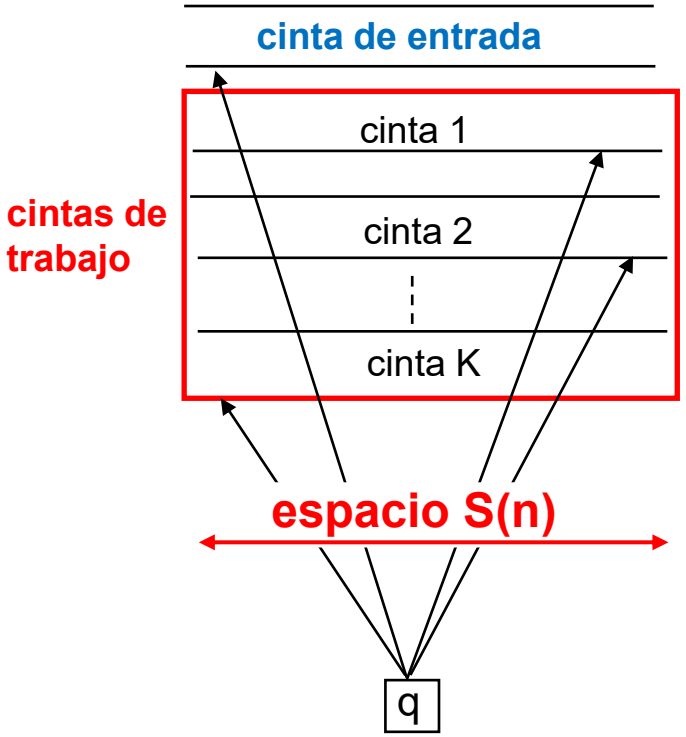
Complejidad espacial



M ocupa a lo sumo S(|w|) celdas, sin considerar la cinta de entrada.

- Una MT M ocupa **espacio S(n)** sii al ejecutarse desde toda entrada w, con |w| = n, M ocupa a lo sumo S(n) celdas en cualquier cinta **distinta de la de entrada**.

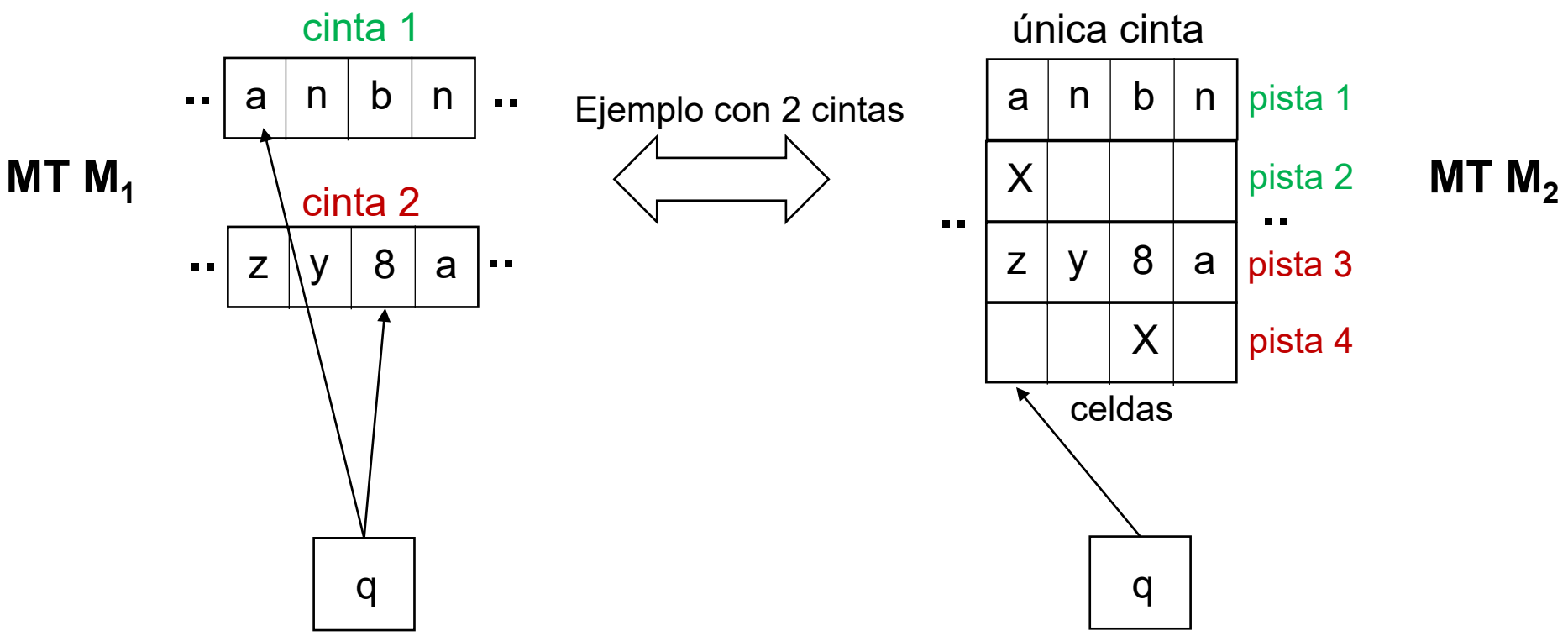
- La cinta de entrada es de **sólo lectura y no se considera en la medición del espacio**. Esto permite un espacio **menor que lineal**, es decir menor que O(n) (la cadena de entrada mide n).



Dada una MT M_1 de espacio S(n) con varias cintas de trabajo, existe una MT M_2 equivalente de espacio S(n) con una sola cinta de trabajo.

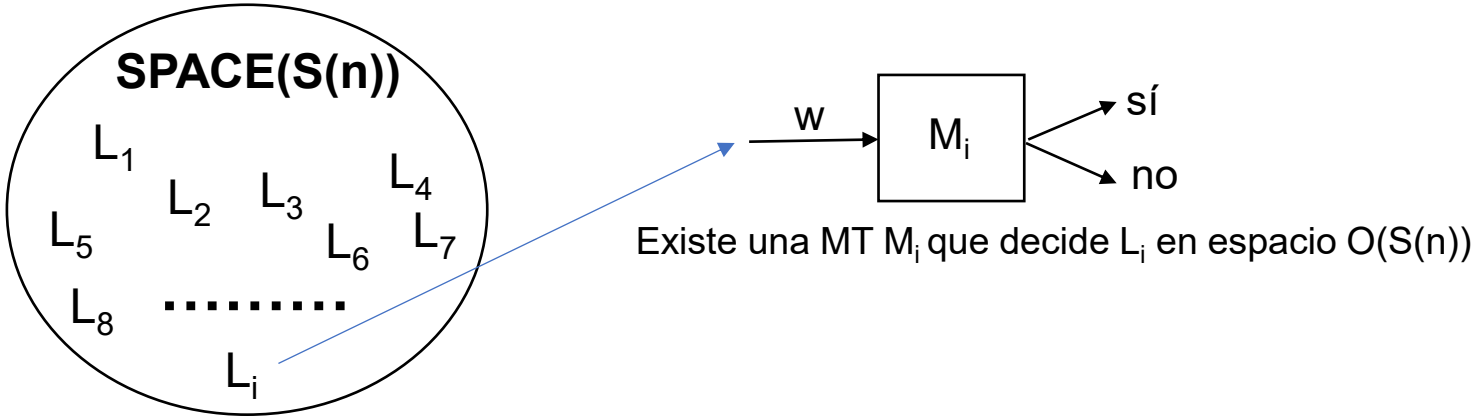
- En el caso más general con una cinta de salida, ésta **tampoco se considera en la medición del espacio.**

Recordar la simulación de una MT con varias cintas mediante una MT con 1 cinta



- Uso de **cintas con pistas**.
- i pasos de M_1 se pueden simular con unos i^2 pasos de M_2 (**retardo cuadrático**).
- j celdas de M_1 se pueden simular con j celdas de M_2 (**mismo espacio**).

- Un lenguaje pertenece a la clase **SPACE(S(n))** sii existe una MT M que lo decide en espacio **O(S(n)).**



1) Una MT M de **tiempo T(n)** ocupa a lo sumo espacio **T(n) celdas**. ¿Por qué? T(n) pasos ≤ T(n) celdas.

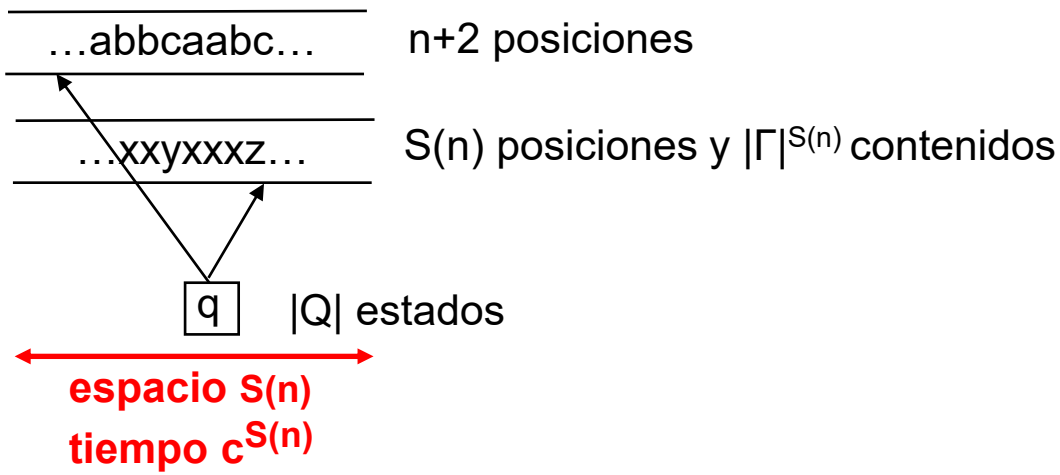
2) En cambio, una MT M que ocupa **espacio S(n)** puede tardar **mucho más que tiempo S(n)**:

Por ejemplo, si M tiene:

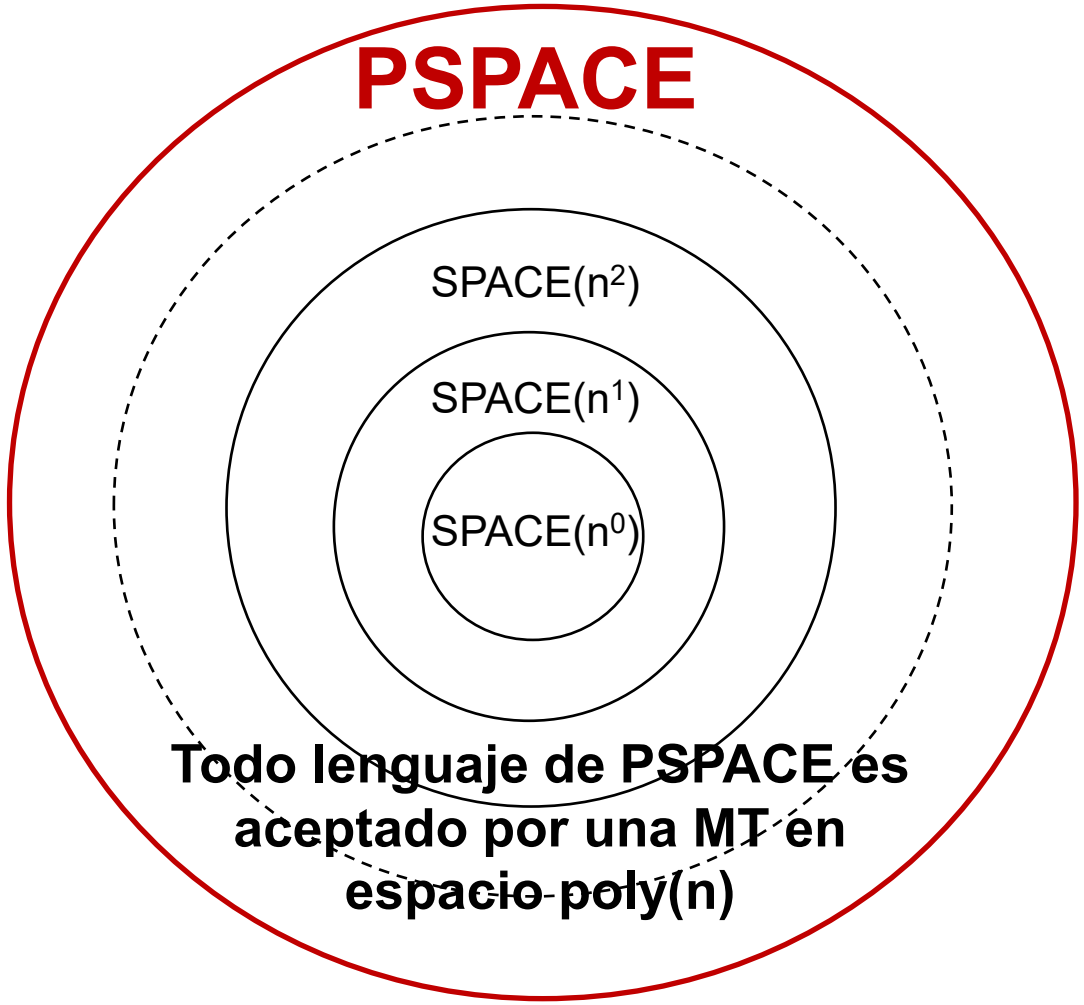
- una cinta de entrada de solo lectura
- una cinta de trabajo
- |Q| estados
- |Γ| símbolos

M puede llegar a hacer, si no loopea:

$$(n+2).S(n).|Q|.|\Gamma|^{S(n)} \leq c^{S(n)} = \exp(S(n)) \text{ pasos}.$$



La clase PSPACE



Robustez: si una MT M_1 con K_1 cintas ocupa espacio poly(n), existe una MT M_2 equivalente con K_2 cintas que también ocupa espacio poly(n).

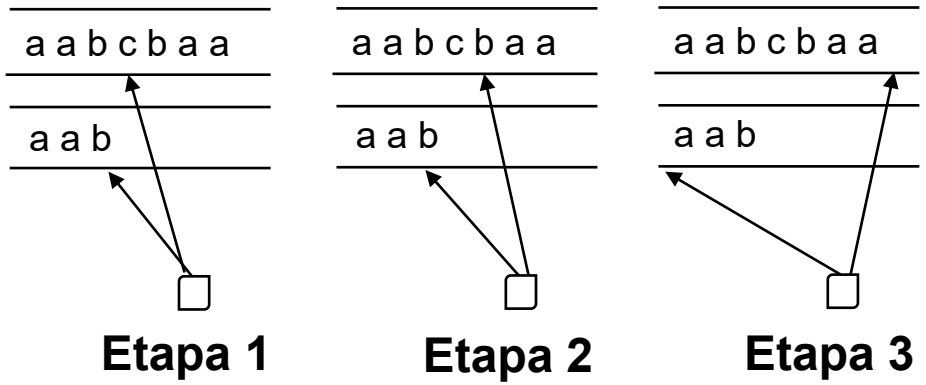
Ejemplo de lenguaje en la clase PSPACE

L = {vcv^R | v^R es la inversa de v y tiene símbolos *a* y b}

La siguiente MT M decide L en **espacio O(n):**

1. Copia la entrada hasta la c exclusive en la cinta 2.
2. Saltea la c en la cinta 1.
3. Compara símbolo a símbolo las dos cintas, yendo a la derecha en la cinta 1 y a la izquierda en la cinta 2. hasta llegar a un blanco en las dos cintas, en cuyo caso acepta.

Por ejemplo:



En la cinta 2, M ocupa espacio O(n)

La clase LOGSPACE

- En realidad, **L = {vcv^R | v^R es la inversa de v y tiene símbolos *a* y b}** se puede decidir en menos espacio:

- La siguiente MT decide L en espacio **O(log$_2$n).** Veámoslo por ejemplo con la entrada **w = aabbbcbbbaa**:

1. Hace i := 1 en la cinta 1.
2. Hace j := n en la cinta 2. Si j es par, rechaza.
3. Copia el símbolo i de w en la cinta 3.
4. Copia el símbolo j de w en la cinta 4.
5. Si i = j: si los símbolos son **c,** acepta, si no, rechaza.
   Si i ≠ j: si los símbolos no son los dos ***a*** o los dos **b**, rechaza.
6. Hace i := i + 1 en la cinta 1.
7. Hace j := j – 1 en la cinta 2.
8. Vuelve al paso 3.

1ra iteración

	a a b b b c b b b a a
1	i = 1
2	j = 11
3	a
4	a

2da iteración

	a a b b b c b b b a a
1	i = 2
2	j = 10
3	a
4	a

.....

anteúltima iteración

	a a b b b c b b b a a
1	i = 5
2	j = 7
3	b
4	b

última iteración

	a a b b b c b b b a a
1	i = 6
2	j = 6
3	c
4	c

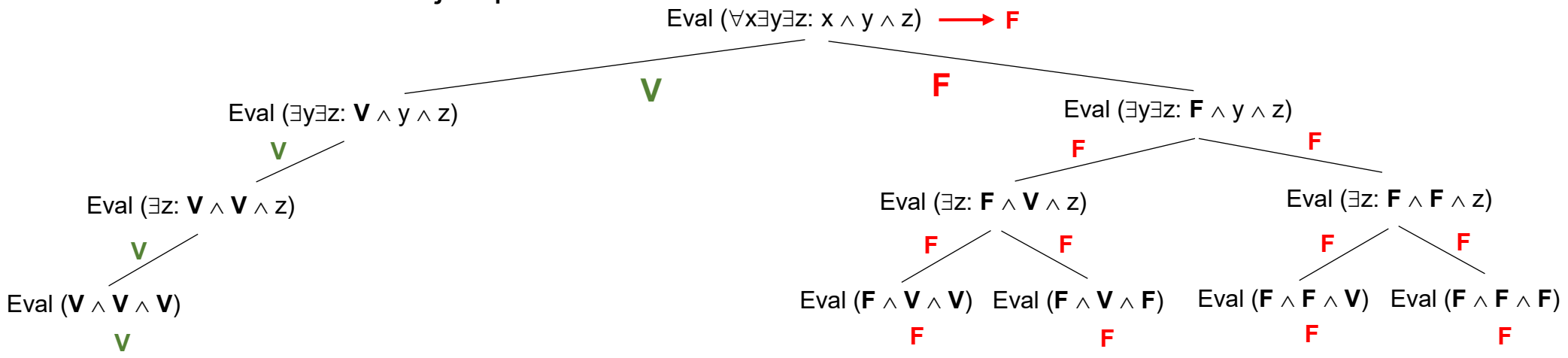
La MT M ocupa el espacio de los contadores i y j, que en binario miden **O(log$_2$n)**, más 2 celdas para alojar cada vez a los símbolos comparados (**espacio constante**).
Por lo tanto, **L ∈ SPACE(log$_2$n).**
A esta clase se la denomina **LOGSPACE.**

Otro ejemplo de lenguaje en la clase PSPACE

QSAT = {φ | φ es una fórmula booleana con cuantificadores, no tiene variables libres, y es verdadera}.

Por ejemplo: $\varphi_1 = \exists x \exists y \exists z: x \wedge y \wedge z$ (fórmula verdadera)
$\varphi_2 = \forall x \exists y \exists z: x \wedge y \wedge z$ (fórmula falsa)

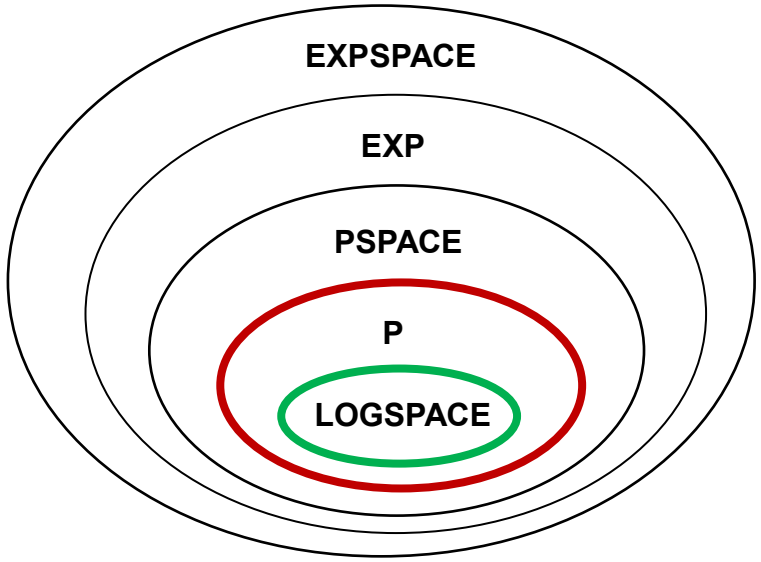
- **QSAT está en PSPACE (y no estaría en NP).** La prueba de que QSAT está en PSPACE se basa en una **función recursiva** Eval. Por ejemplo:



- El espacio que ocupa una rama de la recursión es **reutilizado** por la rama siguiente.
- La profundidad de la recursión es **O(n)** (a lo sumo hay tantas invocaciones como símbolos en la fórmula).
- El tamaño de cada instancia invocada es **O(n)**.
- Por lo tanto, el espacio total ocupado es **$O(n^2)$.**

La jerarquía espacio-temporal (versión 1)

- **LOGSPACE** es la clase de los lenguajes decidibles en espacio **$O(\log_2 n)$**
- **PSPACE** es la clase de los lenguajes decidibles en espacio **poly(n)**
- **EXPSPACE** es la clase de los lenguajes decidibles en espacio **exp(n)**



De antes: **espacio S(n) implica tiempo $c^{S(n)}$**, con c constante

En particular: **espacio $\log_2 n$ implica tiempo $c^{\log_2 n}$**, con c constante

Pero: $c^{\log_2 n} = n^{\log_2 c}$, con c constante, es decir: $c^{\log_2 n} = poly(n)$

por lo tanto: **espacio $\log_2 n$ implica tiempo poly(n)**

Se conjetura que LOGSPACE $\subset$ P

Se conjetura que P $\subset$ PSPACE

Se prueba que LOGSPACE $\subset$ PSPACE

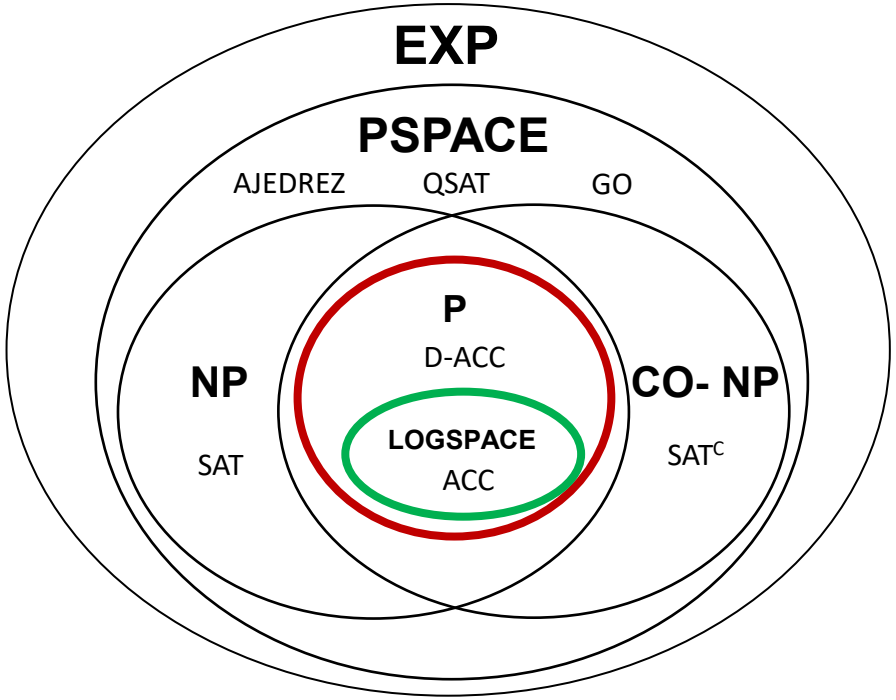
Se conjetura que PSPACE $\subset$ EXP

Se prueba que EXP $\subset$ EXPSPACE

En definitiva, **LOGSPACE $\subseteq$ P**,
es decir que **los lenguajes de LOGSPACE son tratables**.

Anexo 1

La jerarquía espacio-temporal (versión 2)



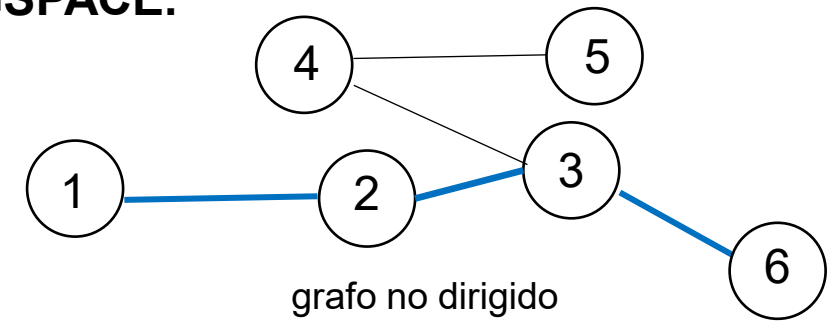
Ejemplos

- QSAT está entre los lenguajes más difíciles de PSPACE.
  Es **PSPACE-completo** (todo L ∈ PSPACE cumple $L \leq_p$ QSAT).

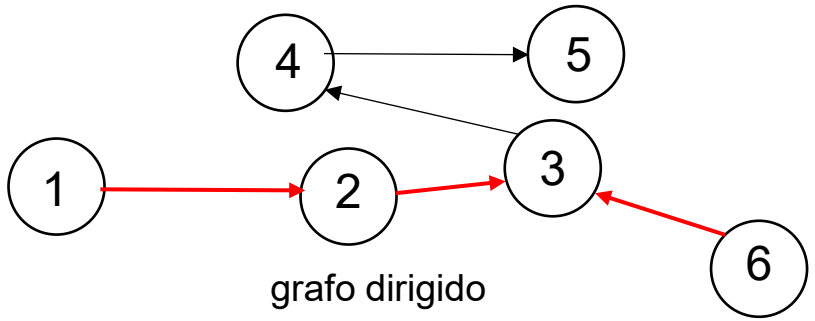
- Otros problemas **PSPACE-completos** se asocian a **juegos entre dos jugadores J_1 y J_2**. Hay que decidir, dada una posición, si hay una jugada de J_1 tal que para toda jugada de J_2 hay una jugada de J_1 tal que para toda jugada de J_2 … J_1 gana (**ajedrez, go, etc).**

- **ACC** = {G | G es un grafo no dirigido con un camino del vértice 1 al vértice m} **está en LOGSPACE.**
  No se copia todo G en una cinta sino que se va copiando de a un vértice por vez.

- **D-ACC** = {G | G es un grafo dirigido con un camino del vértice 1 al vértice m} **no estaría en LOGSPACE, pero sí en NLOGSPACE.**



grafo no dirigido

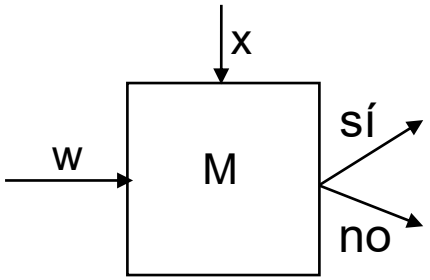


grafo dirigido

En el segundo caso hay que contemplar la dirección de los arcos

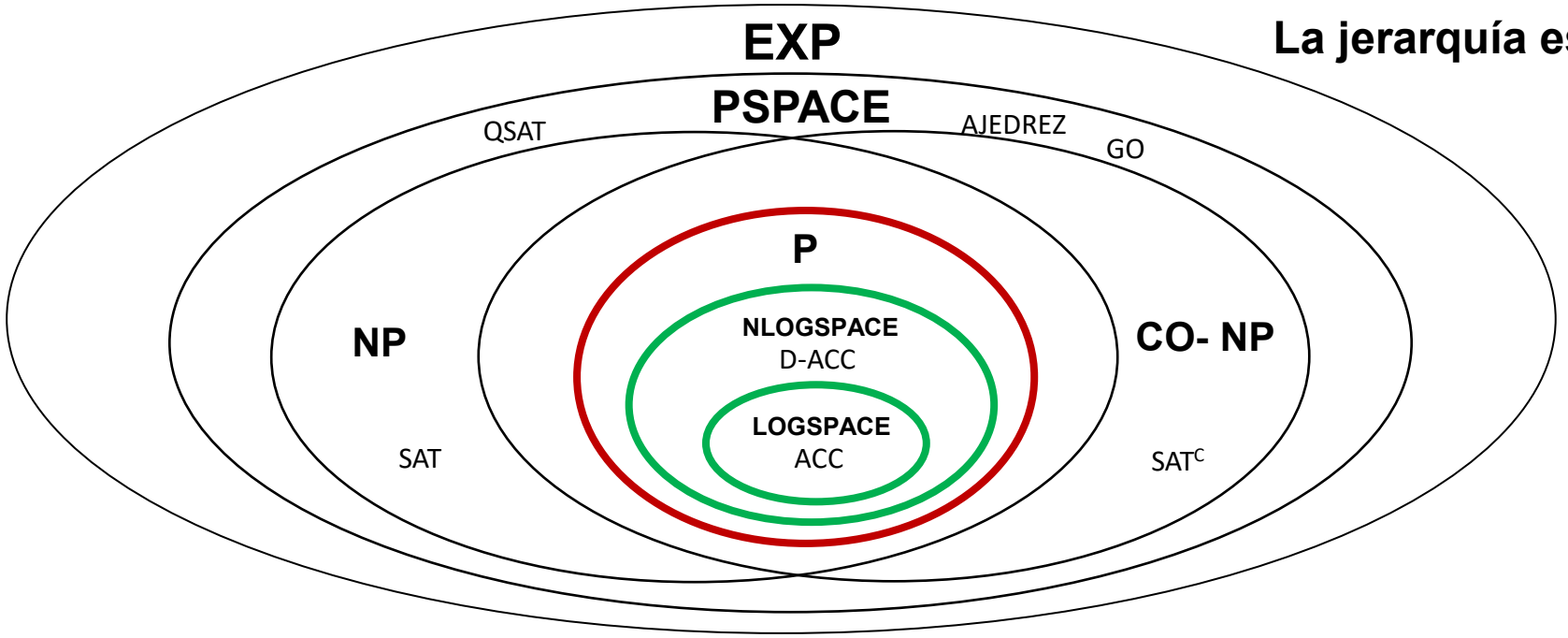
La clase NLOGSPACE

- Un lenguaje L está en la clase NLOGSPACE si existe una MT M que lo **verifica** en espacio **$O(\log_2|w|)$,** con la ayuda de un **certificado sucinto x**, es decir $|x| \leq poly(|w|)$, tal que x se recorre sólo hacia la derecha.



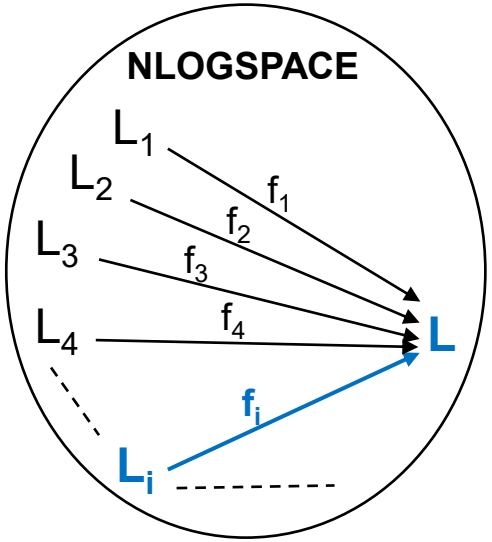
w está en la cinta de entrada, de sólo lectura.
x está en una cinta auxiliar, de sólo lectura, **y se recorre sólo hacia la derecha** (así se evita el espacio polinomial).

La jerarquía espacio-temporal (versión 3)

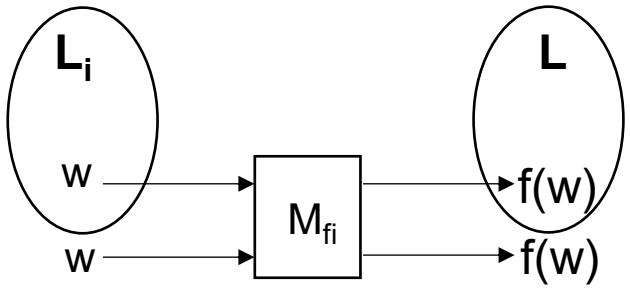


La conjetura aceptada es
LOGSPACE $\subset$ NLOGSPACE $\subset$ P

- **Si un lenguaje L es NLOGSPACE-completo y está en LOGSPACE, entonces LOGSPACE = NLOGSPACE.**

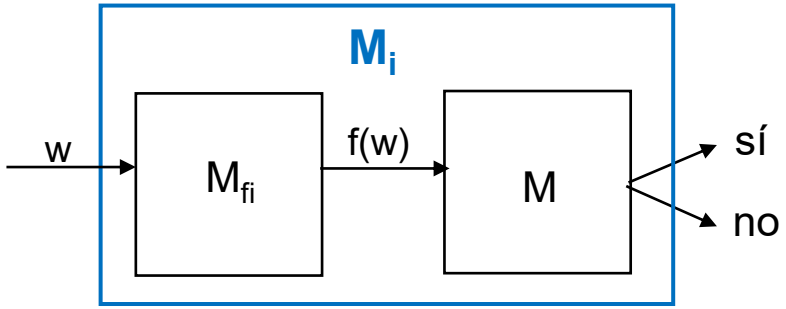


Ahora las f_i son **reducciones log-space** (ocupan espacio logarítmico)



M_{fi} computa f_i en espacio $O(\log_2|w|)$ ($|w|$ y $|f(w)|$ no cuentan)

Si M es una MT que decide L en espacio logarítmico, la siguiente MT M_i decide L_i en espacio logarítmico:



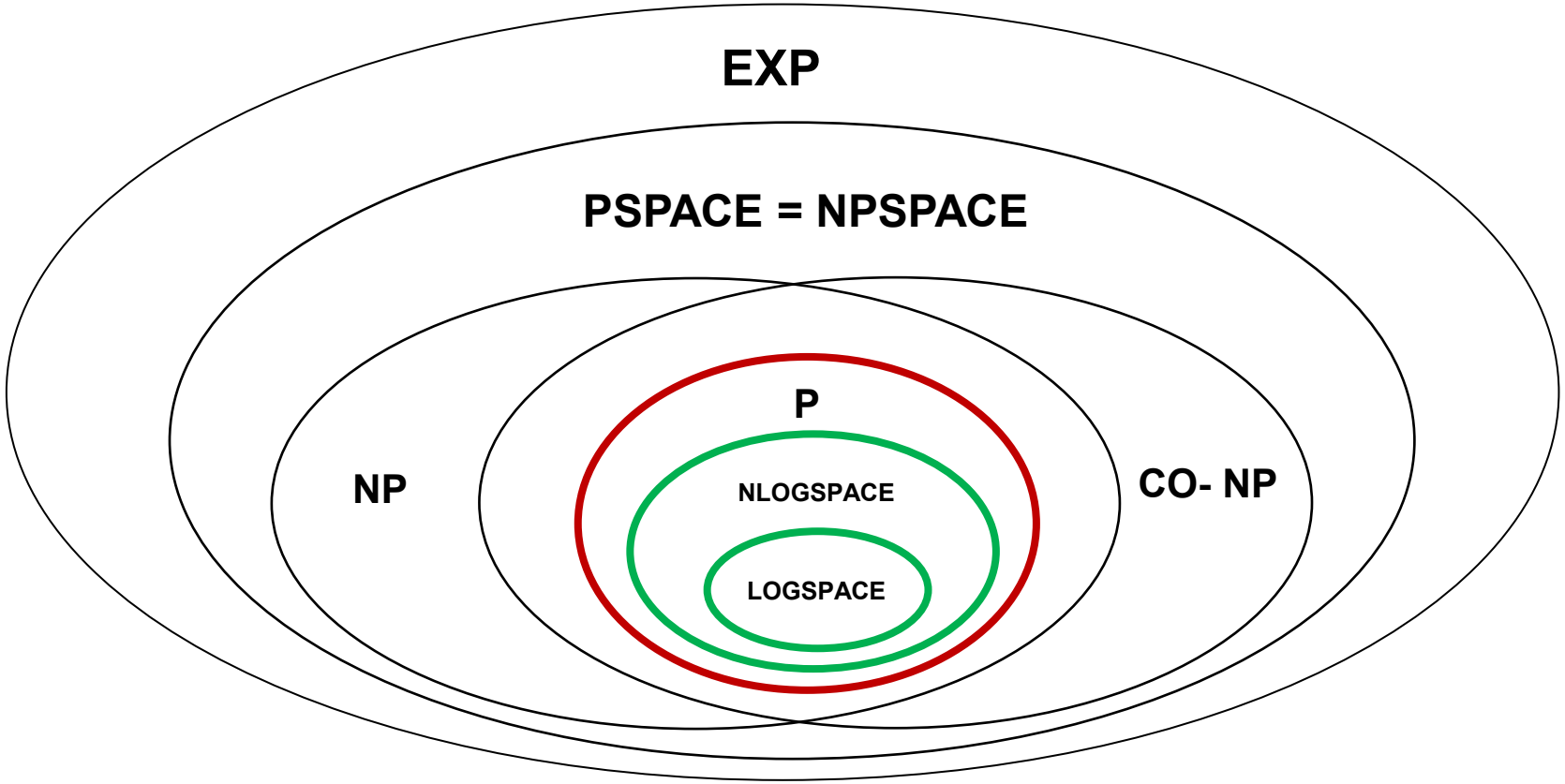
Composición de MT de espacio logarítmico
M_{fi} computa f_i en espacio logarítmico
M decide L en espacio logarítmico
¡Pero f(w) puede medir poly(|w|)! (¿por qué?)
Porque espacio log(n) implica tiempo poly(n)

Se puede construir M_i para que no viole la cota espacial logarítmica:

M_{fi} no le envía f(w) completo a M, sino que le pasa un símbolo cada vez, el que M requiere.

- La clase **NPSPACE** se define de manera similar a cómo se definen NP y NLOGSPACE. Se cumple **PSPACE = NPSPACE**.

La jerarquía espacio-temporal (versión 4)



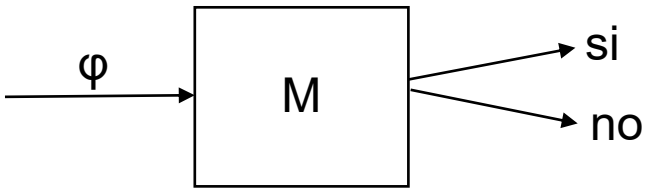
Se prueba que LOGSPACE $\subset$ PSPACE

Se prueba que P $\subset$ EXP

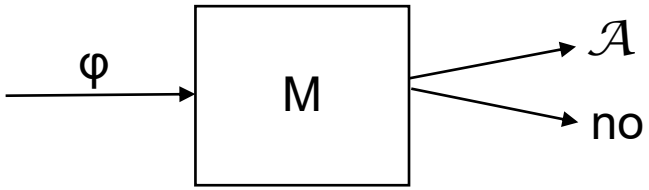
Anexo 2

Complejidad temporal de los problemas de búsqueda

- Los problemas más generales son los de **búsqueda**.
- Las MT asociadas **devuelven una solución** si existe.
- **Ejemplo. Problema FSAT**: problema de búsqueda asociado a SAT.



MT que resuelve SAT



MT que resuelve FSAT

Por ejemplo:

$\varphi = (x_1 \vee x_2) \wedge (x_3 \wedge x_4)$

$\mathcal{A} = (V, F, V, V)$

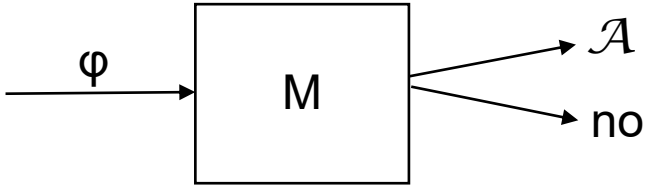
- Las clases de problemas de búsqueda asociadas a las clases de lenguajes P, NP, etc., se identifican con **FP, FNP,** etc. Por ejemplo, **FSAT ∈ FNP**.
- Intuitivamente, **decidir la existencia** de una solución es más fácil que **encontrar una.** Pero en el caso, por ejemplo, de los lenguajes **NP-completos**, ¿realmente es así?

Mostramos que **no** con un ejemplo.

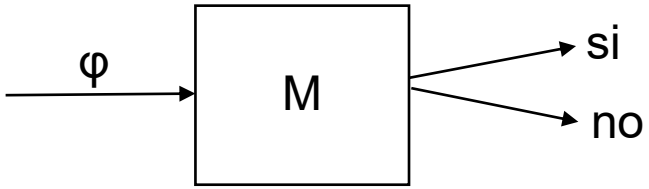
- **Intuitivo: El problema FSAT (búsqueda) es tan o más difícil que el problema SAT (decisión).**

Si existe una MT M_{FSAT} que resuelve FSAT en tiempo poly(n),

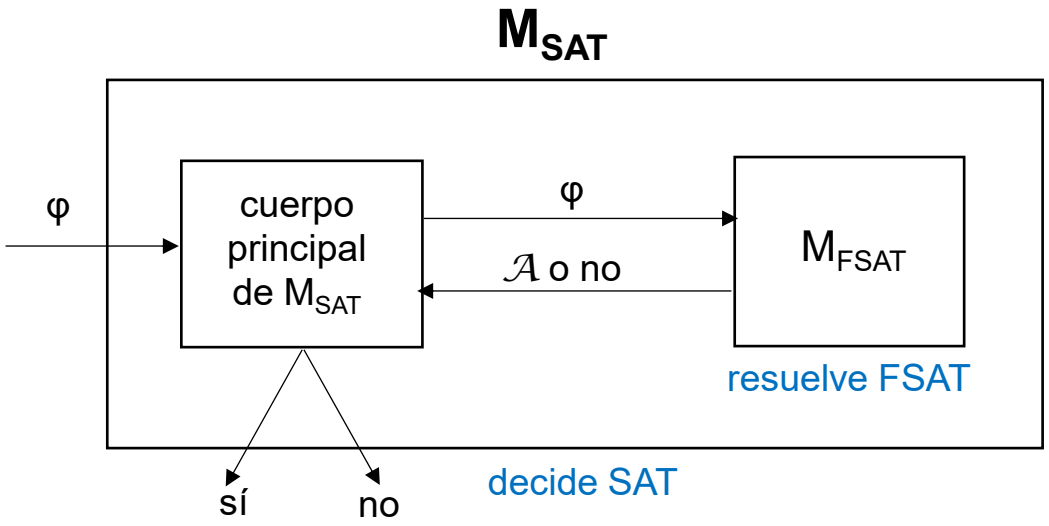
entonces existe una MT M_{SAT} que resuelve SAT en tiempo poly(n).



MT que resuelve FSAT



MT que resuelve SAT

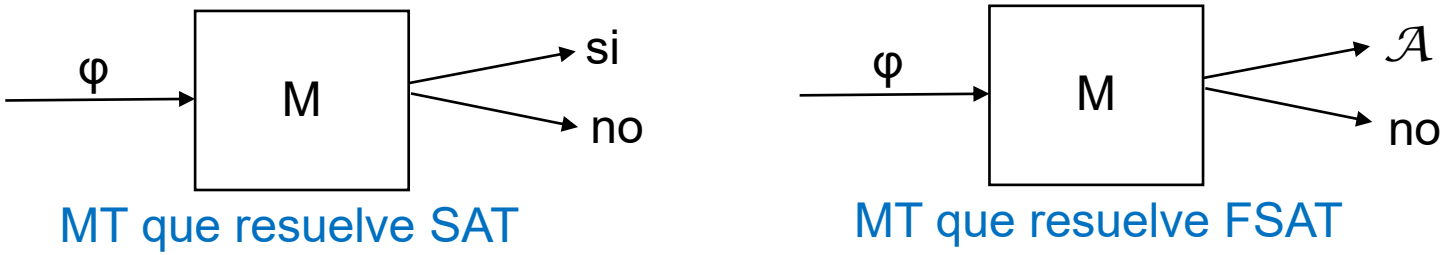


Si FSAT ∈ FP entonces SAT ∈ P.
O lo mismo: si SAT ∉ P entonces FSAT ∉ FP.

FSAT es tan o más difícil que SAT

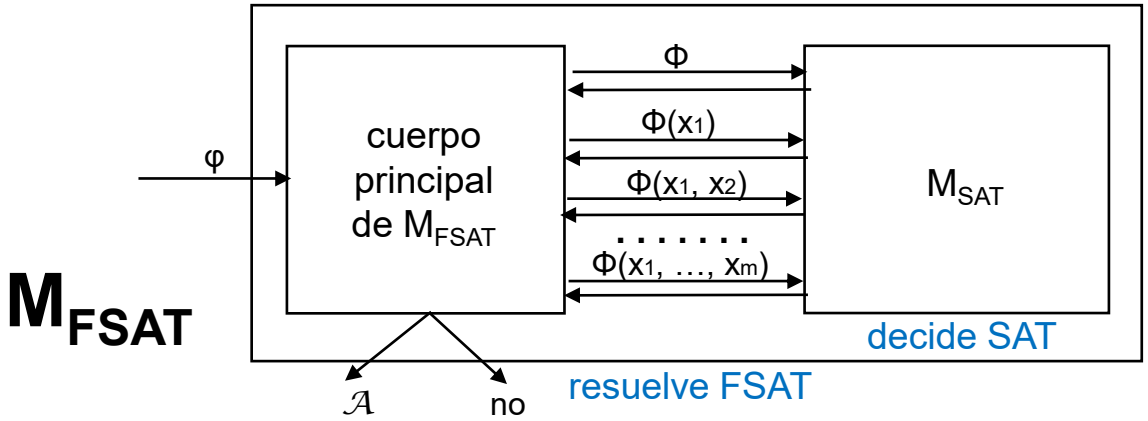
- **No intuitivo: El problema SAT (decisión) es tan o más difícil que el problema FSAT (búsqueda).**

Si M_{SAT} es una MT que resuelve SAT en tiempo poly(n), existe una MT M_{FSAT} que resuelve FSAT en tiempo poly(n):



Dada una fórmula φ con m variables, M_{FSAT} hace:

1. Invoca a M_{SAT} con φ. Si M_{SAT} responde no, entonces **responde no**.
2. Invoca a M_{SAT} con φ haciendo x_1 = verdadero.
   Si M_{SAT} responde sí, **fija para x_1 el valor verdadero**, y si responde no, **fija para x_1 el valor falso**.
3. Invoca a M_{SAT} con φ, utilizando el valor de x_1 obtenido y haciendo x_2 = verdadero.
   Si M_{SAT} responde sí, **fija para x_2 el valor verdadero**, y si responde no, **fija para x_2 el valor falso**.
4. Repite el proceso con las variables **x_3, x_4, …, x_m**, y al final **devuelve la asignación $\mathcal{A}$ obtenida**.



Si SAT ∈ P, FSAT ∈ FP.
Lo mismo: si FSAT ∉ FP, SAT ∉ P.

SAT es tan o más difícil que FSAT

Clase práctica 7

Ejemplo. Prueba de la NP-completitud del problema del cubrimiento de vértices

- Probaremos que CV = {(G, K) | G es un grafo no dirigido que tiene un cubrimiento de vértices de tamaño K} es NP-completo.

- Proponemos la siguiente reducción polinomial de 3-SAT (NP-completo) a CV (se prueba fácil que está en NP):

$$\mathbf{f(\varphi) = (G, 2K)}$$

tal que φ es una fórmula booleana en FNC con tres literales por cláusula, K es la cantidad de cláusulas de φ, y G es un grafo que se construye de la siguiente forma:

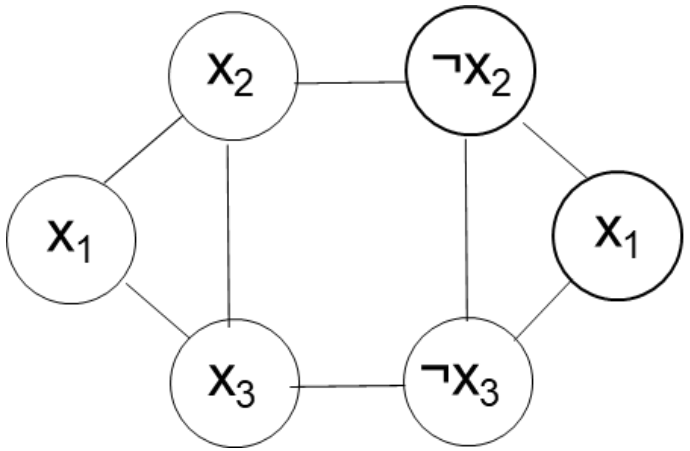
1. Por cada literal de cada cláusula de φ se crea un vértice en G.

2. Todo par de vértices creados a partir de dos literales de una misma cláusula se unen por un arco (identificado como *enlace de tipo 1*). De esta manera, toda cláusula determina un triángulo con enlaces de tipo 1.

3. Todo par de vértices creados a partir de dos literales x_i y $\neg x_i$ de distintas cláusulas también se unen por un arco (identificado como *enlace de tipo 2*).

(sigue)

- Por ejemplo, el grafo G siguiente se obtiene a partir de la fórmula booleana $\boldsymbol{\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3)}$:



- Notar que $\boldsymbol{\{x_2, x_3, \neg x_2, \neg x_3\}}$ es un **cubrimiento de vértices de tamaño 4 de G,** el doble de la cantidad de cláusulas de φ, tal como se requiere.

- La función f es efectivamente una **reducción polinomial de 3-SAT a CV.** En tiempo polinomial, por cada una de las K cláusulas se puede crear un triángulo, y por cada literal de cada cláusula se pueden crear O(K) arcos.

- Y se cumple $\boldsymbol{\varphi \in}$ **3-SAT sii (G, 2K) ∈ CV:**

(sigue)

- **Sean φ ∈ 3-SAT, $\mathcal{A}$ una asignación que satisface φ, y el siguiente conjunto C de 2K vértices de G:**

  Incluye a todos los vértices asociados a los literales falsos de φ según $\mathcal{A}$, y si con esto no es suficiente para que tenga dos vértices de cada triángulo (por existir cláusulas de φ con dos o tres literales verdaderos según $\mathcal{A}$), incluye más vértices hasta completar el par.

  Se cumple que C es un cubrimiento de vértices de G: todo enlace de tipo 1 está cubierto porque C tiene dos vértices de cada triángulo formado con dichos enlaces, y todo enlace de tipo 2 también está cubierto porque uno de sus vértices está asociado a un literal falso según $\mathcal{A}$, el cual está en C.

  Así, (G, 2K) ∈ CV.

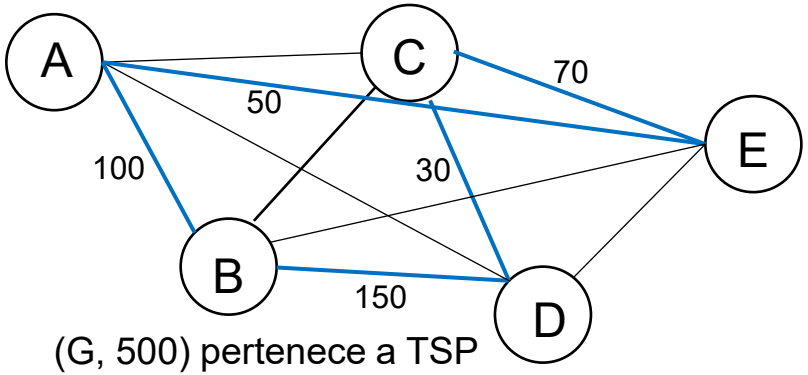
- **Sean (G, 2K) ∈ CV, C un cubrimiento de vértices de tamaño 2K de G, y la siguiente asignación $\mathcal{A}$ a φ:**

  A los literales asociados a los vértices que no están en C les asigna el valor *verdadero*, y al resto de los literales les asigna valores consistentes cualesquiera.

  Se cumple que $\mathcal{A}$ satisface φ: C está compuesto necesariamente por dos vértices de cada uno de los triángulos, y por lo tanto en toda cláusula hay un literal verdadero según $\mathcal{A}$. Además, $\mathcal{A}$ no tiene inconsistencias, porque si dos literales x_i y $\neg x_i$ de dos cláusulas distintas son verdaderos según $\mathcal{A}$, significa que el enlace de tipo 2 asociado no está cubierto por C (contradicción).

  Así, φ ∈ 3-SAT.

Ejemplo. El viajante de comercio revisitado.



TSP = {(G, B) | G es un grafo completo ponderado (arcos con números) y tiene un circuito de Hamilton (CH) cuyos arcos suman ≤ B}.

OTSP: "Hallar un CH mínimo de un grafo completo ponderado G".

<u>Se cumple que OTSP es tan o más difícil que TSP (intuitivo):</u>

Es decir, si se puede encontrar en tiempo poly(n) un CH mínimo de un grafo completo ponderado G, entonces también se puede decidir en tiempo poly(n) si G tiene un CH cuyos arcos suman ≤ B (ejercicio).

<u>También se cumple que TSP es tan o más difícil que OTSP (menos intuitivo):</u>

Es decir, si se puede decidir en tiempo poly(n) si un grafo completo ponderado G tiene un CH cuyos arcos suman ≤ B (MT M_{TSP}), también se puede encontrar en tiempo poly(n) un CH mínimo de G (MT M_{OTSP}).

La MT M_{OTSP} referida se comporta de la siguiente manera. Dado un grafo completo ponderado G, hace:

Paso 1. Obtiene, invocando a la MT M_{TSP} , la suma **N** de los arcos de un CH mínimo de G.
Paso 2. Utilizando N, encuentra un **CH mínimo** de G.

Sigue en la hoja siguiente

Paso 1. Obtención de la suma N de los arcos de un CH mínimo de G:

- M_{OTSP} invoca a M_{TSP} con cadenas (G, N) varias veces, **sin modificar G pero modificando N:**
- Como el valor de N, codificado en binario, es a lo sumo 2^n siendo n = |G| (¿por qué?), entonces por **búsqueda binaria**, arrancando por ejemplo con la cadena (G, $2^n/2$), N se puede obtener luego de $O(\log_2 2^n)$ = **O(n) invocaciones** (¿qué hace M_{OTSP} en cada iteración?)

Paso 2. Obtención de un CH mínimo de G utilizando N:

- M_{OTSP} invoca a M_{TSP} con cadenas (G, N) varias veces, ahora **sin modificar N pero modificando G:**
- En cada invocación, **modifica el número de un arco distinto de G, asignándole el valor N + 1:**
  Si M_{TSP} acepta, significa que dicho arco no forma parte de un CH mínimo, y por lo tanto no hace nada.
  Si M_{TSP} rechaza, significa que dicho arco forma parte de un CH mínimo, así que en este caso **marca el arco** (compondrá el CH mínimo que va a devolver) **y le restituye su número original**.
- M_{OTSP} procesa de esta manera todos los arcos, y por lo tanto efectúa |E| = **O(n) invocaciones**.

En verdad, si G tiene más de un CH mínimo, M_{TSP} en el paso 2 puede aceptar aún cuando el arco modificado forme parte de un CH mínimo. ¿Por qué de todos modos el algoritmo es correcto?

Claramente, M_{OTSP} resuelve OTSP y lo hace en tiempo poly(n)